08 | 工程初始化（上）：后端骨架与公共基础设施

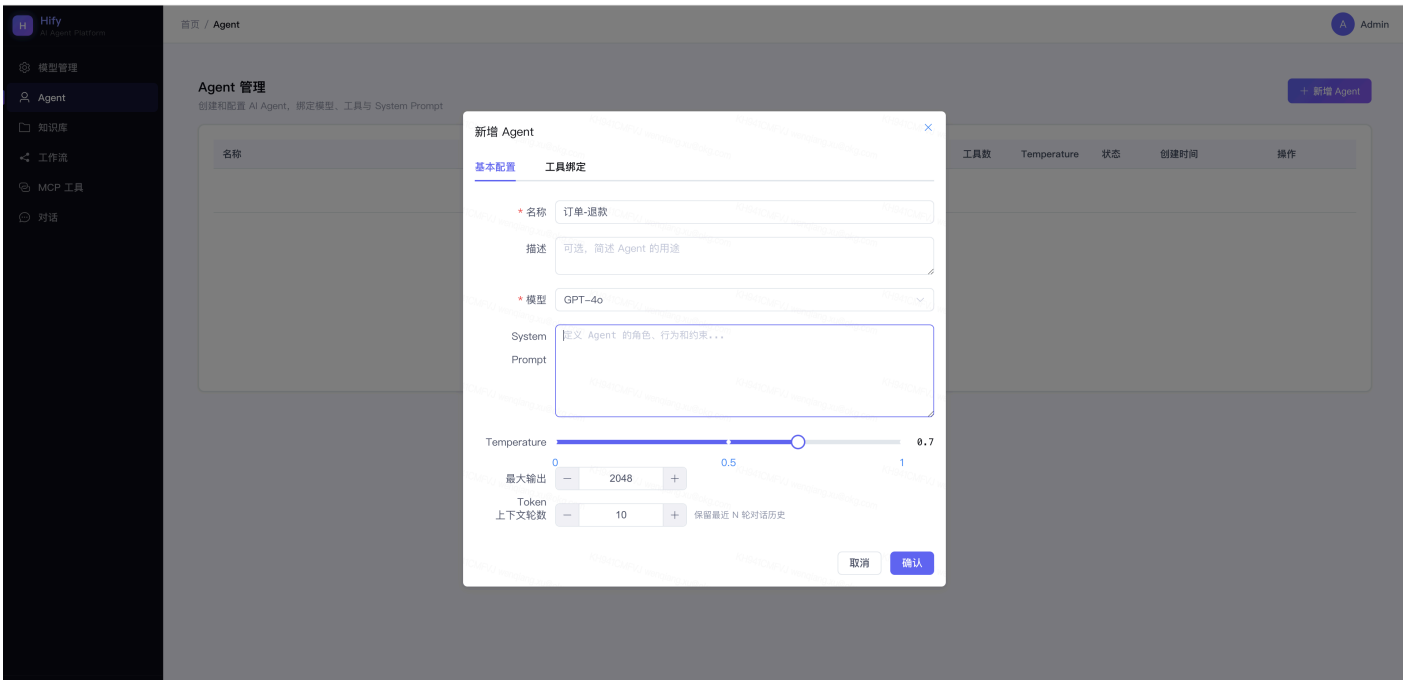
Robert · Claude Code 企业级全链路开发实战



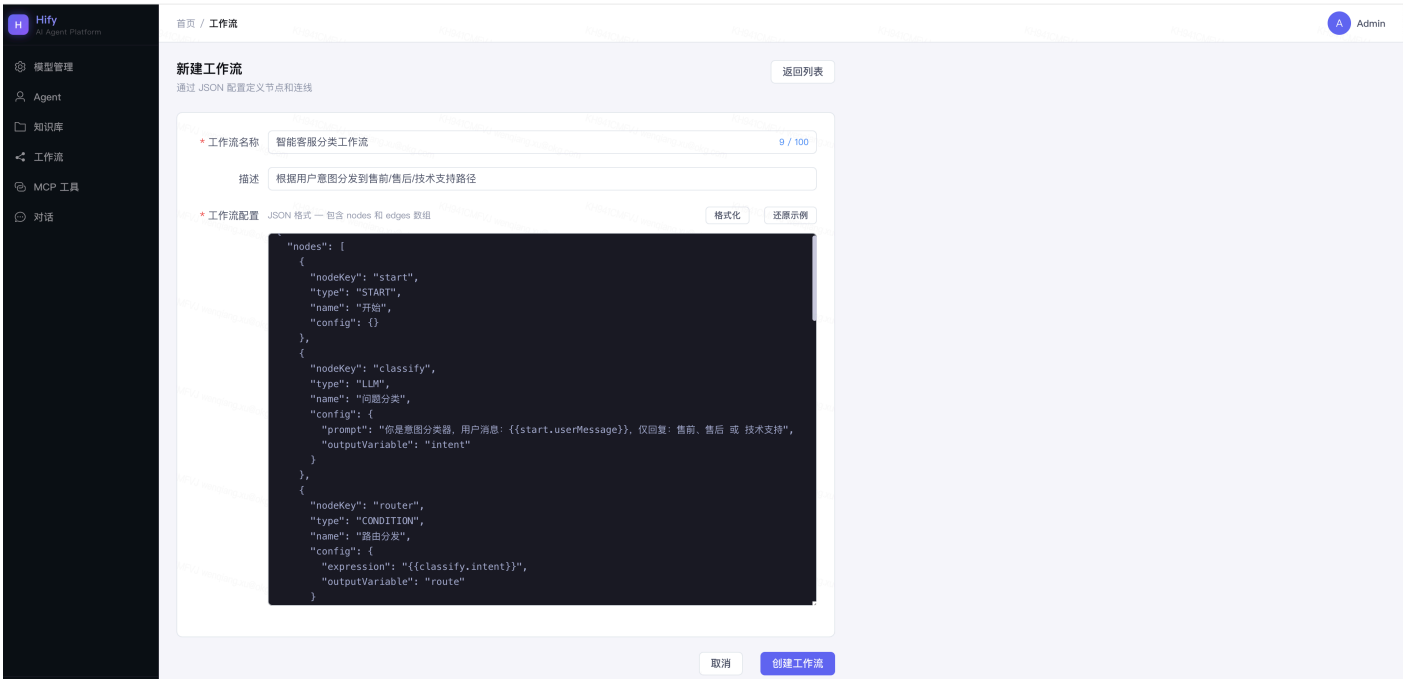
你好，我是 Robert。

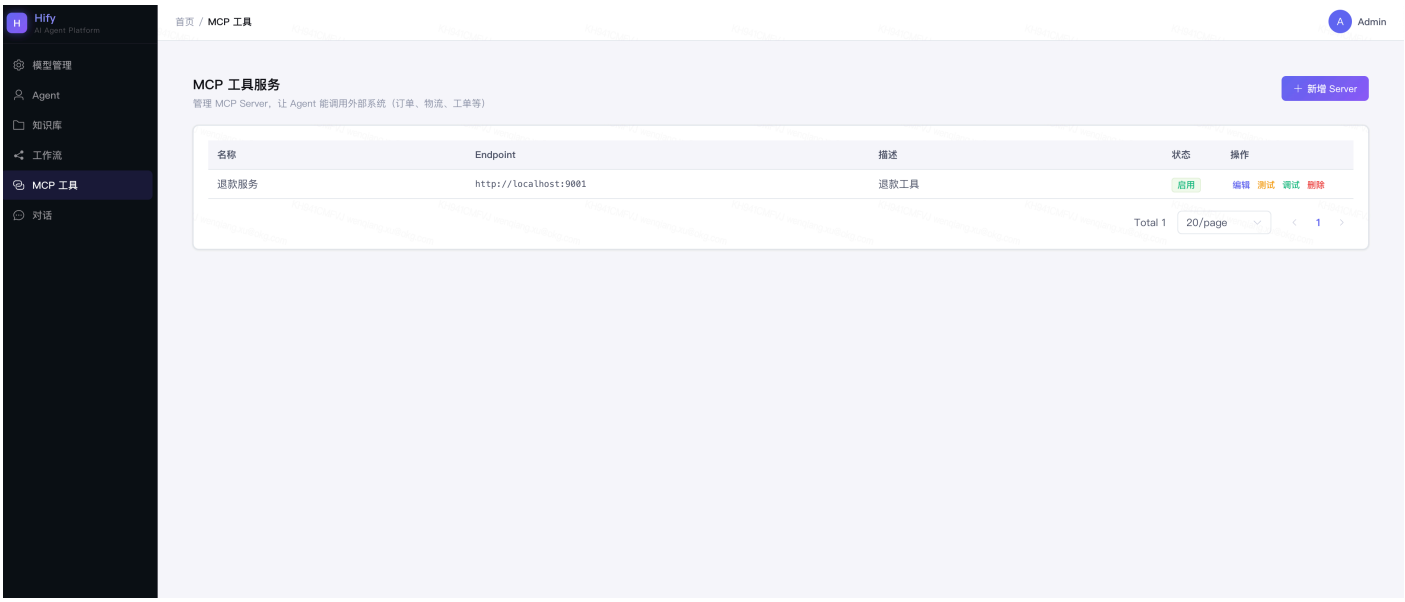
从这节课开始，我们正式进入工程部分。先来展示下 Hify 的本地工作界面吧，让你知道我们即将完成的软件系统是什么样子的。

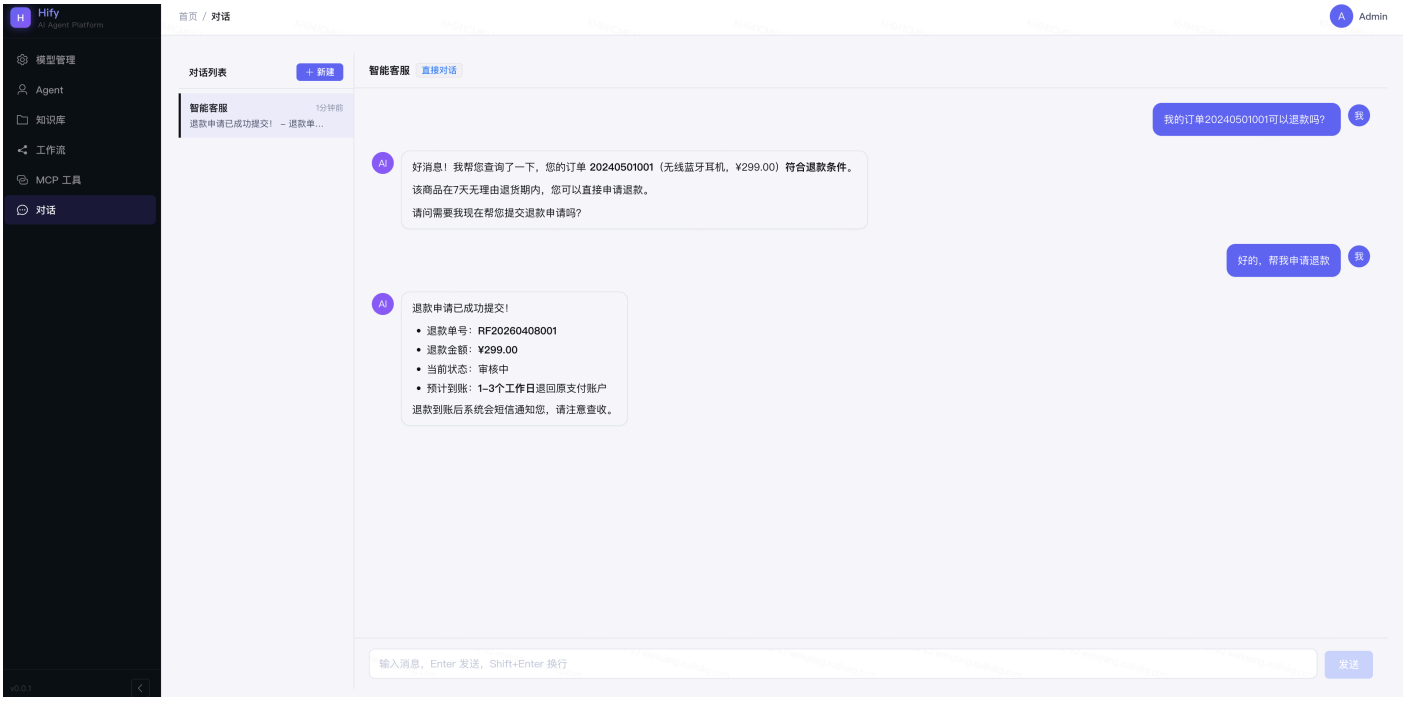

Hify
AI Agent Platform
首页 / 模型管理
Admin
模型管理
Agent
知识库
工作流
MCP 工具
对话
模型提供商管理
管理接入的 AI 模型提供商，配置 API Key 和连接信息
+ 新增提供商
名称
状态
健康
模型数
操作
Mock
启用
未知
0 个
编辑 测试 删除
OpenAI
启用
未知
1 个
编辑 测试 删除
Total 2
20/page
1
提供商信息
* 名称
Claude Code
* 类型
Anthropic (Claude)
API Key
留空表示不修改
* Base URL
https://claude.ai
描述
可选
取消
确认


Hify
AI Agent Platform
首页 / Agent
Admin
模型管理
Agent
知识库
工作流
MCP 工具
对话
Agent 管理
创建和配置 AI Agent，绑定模型、工具与 System Prompt
+ 新增 Agent
名称
工具数
Temperature
状态
创建时间
操作
新增 Agent
基本配置
工具绑定
* 名称
订单-退款
描述
可选，简述 Agent 的用途
* 模型
GPT-4o
System Prompt
定义 Agent 的角色、行为和约束...
Temperature
0.7
0
0.5
1
最大输出
2048
Token 上下文轮数
10
保留最近 N 轮对话历史
取消
确认


Hify
AI Agent Platform
首页 / 知识库
创建成功
Admin
模型管理
Agent
知识库
工作流
MCP 工具
对话
知识库管理
管理 RAG 知识库，上传文档自动分块向量化
搜索知识库名称
+ 新建知识库
启用 编辑 删除
活动奖励
暂无描述
2026/4/8
查看文档 →
启用 编辑 删除
优惠规则
暂无描述
2026/4/8
查看文档 →
启用 编辑 删除
退款规则
暂无描述
2026/4/8
查看文档 →


Hify
AI Agent Platform
模型管理
Agent
知识库
工作流
MCP 工具
对话
首页 / 工作流
Admin
新建工作流
通过 JSON 配置定义节点和连线
返回列表
* 工作流名称
智能客服分类工作流
9 / 100
描述
根据用户意图分发到售前/售后/技术支持路径
* 工作流配置
JSON 格式 — 包含 nodes 和 edges 数组
格式化
还原示例
{
"nodes": [
{
"nodeKey": "start",
"type": "START",
"name": "开始",
"config": {}
},
{
"nodeKey": "classify",
"type": "LLM",
"name": "问题分类",
"config": {
"prompt": "你是意图分类器，用户消息：{{start.userMessage}}，仅回复：售前、售后 或 技术支持",
"outputVariable": "intent"
}
},
{
"nodeKey": "router",
"type": "CONDITION",
"name": "路由分发",
"config": {
"expression": "{{classify.intent}}",
"outputVariable": "route"
}
取消
创建工作流


Hify
AI Agent Platform
模型管理
Agent
知识库
工作流
MCP 工具
对话
首页 / MCP 工具
Admin
MCP 工具服务
管理 MCP Server，让 Agent 能调用外部系统（订单、物流、工单等）
+ 新增 Server
名称
Endpoint
描述
状态
操作
退款服务
http://localhost:9001
退款工具
启用
编辑 测试 调试 删除
Total 1
20/page
1



好，看完了效果，我们开始今天的课程内容。先回顾一下，04 讲我们定了模块化单体、Maven 9 个模块的应用架构，05 讲定了单机部署、线程池隔离、熔断策略的运行架构，06 讲把这一切写进了 CLAUDE.md。蓝图画完了，现在按图施工。

这一讲要做的事：**让 Claude Code 搭建 Hify 后端的工程骨架**。Maven 多模块结构、公共基础设施（统一响应、全局异常处理、MyBatis-Plus 配置、Redis 配置）、业务模块的空壳，全部就位。搭完之后，`java -jar` 能跑起来，访问健康检查接口返回 200。

前端工程和启动脚本放到下一讲。这一讲先把后端的地基打扎实。

先想清楚怎么拆

你可能会想：工程初始化不就是模板性工作吗？把 CLAUDE.md 喂给 Claude Code，说一句“帮我初始化 Hify 后端项目”不就完了？

我试过，效果不好。

Claude Code 一次性生成了几十个文件，看起来很完整。但仔细一看：父 pom 里的模块声明和实际目录不一致，hify-common 的依赖被写进了 hify-chat 的 pom 里，MyBatis-Plus 的

版本和 Spring Boot 的版本有冲突，全局异常处理器和 Result 类用了两种不同的错误码格式。

问题不在 Claude Code 能力不够，是一次性生成的东西太多了，它没法保证所有细节都自洽。几十个文件之间有大量的依赖和关联，一个地方错了会连锁影响其他地方。而且生成量太大，你 review 起来也吃力，容易漏掉问题。

当然随着模型能力越来越强，这个问题也会慢慢解决甚至不存在。但是即使有那一天，我也是希望你能把后面的内容看完。**因为我们要做的是一个可持续迭代的系统，而不是挑战 Claude Code 的能力**。我希望你学到是“道”而不只是“术”。

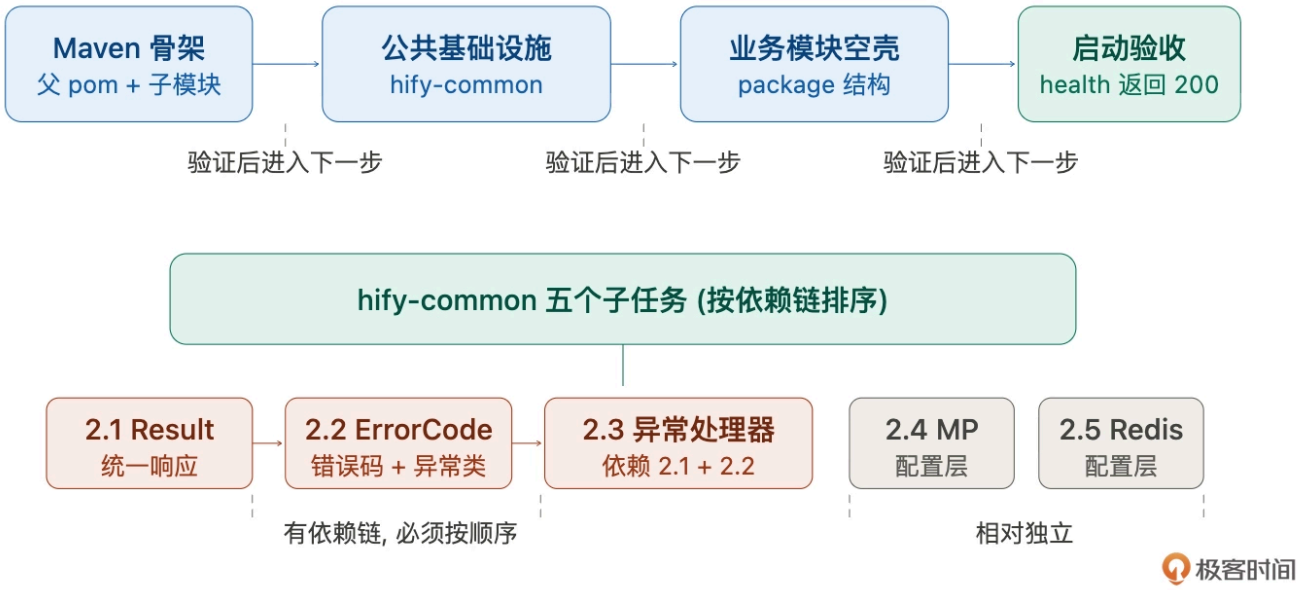
所以第一个方法论在这里出场：**面对体量大的任务，要拆**。

怎么判断该不该拆？两个标准：

1. 标准一：生成的代码量是否超出你一次能 review 的范围。十几行代码，肉眼扫一遍就行，不用拆。几十个文件、几百行配置，一次看不过来，必须拆。
2. 标准二：步骤之间是否有依赖关系。如果第二步依赖第一步的结果（比如业务模块的 pom 依赖公共模块的 pom），那就应该先完成第一步、验证正确，再做第二步。否则第一步错了，第二步会错得更离谱。

工程初始化两个条件都满足。所以我把它拆成四步：

1. Maven 多模块骨架（父 pom + 子模块 pom + 目录结构）
2. hify-common 公共基础设施（Result、BizException、全局异常处理、配置类）
3. 业务模块空壳（每个模块的 package 结构和启动验证）
4. 验收，启动项目，确认一切正常



为什么是这个顺序？依赖关系决定了顺序。Maven 骨架是所有东西的容器，必须先有。hify-common 被所有业务模块依赖，排第二。业务模块依赖 common，排第三。最后验收确认整体能跑。

先地基，后框架，最后验收。这个顺序不只适用于工程初始化，后面做任何模块都是这个思路。

分步搭建

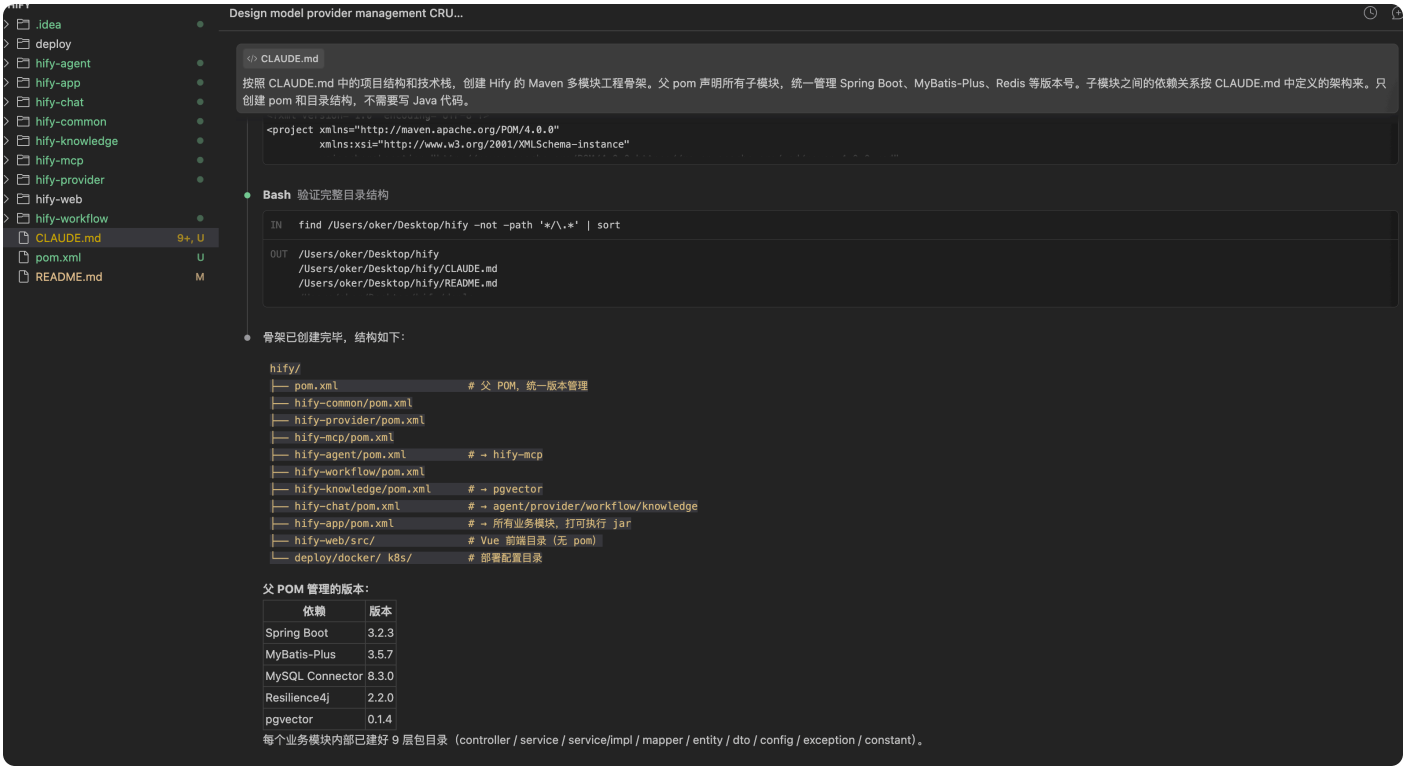
第一步：Maven 多模块骨架

目标很明确：创建父 pom 和所有子模块的 pom，把目录结构搭出来，依赖关系配对。

给 Claude Code 的指令思路：

> 按照 CLAUDE.md 中的项目结构和技术栈，创建 Hify 的 Maven 多模块工程骨架。父 pom 声明所有子模块，统一管理 Spring Boot、MyBatis-Plus、Redis 等版本号。子模块之间的依赖关系按 CLAUDE.md 中定义的架构来。只创建 pom 和目录结构，不需要写 Java 代码。

注意最后一句，“只创建 pom 和目录结构，不需要写 Java 代码”。这是控制范围的关键。不说这句，Claude Code 会顺手给你生成 Application 类、配置文件、甚至示例代码，和后面步骤冲突。



这段代码展示了 Claude Code 生成的父 pom.xml 核心部分（模块声明、版本管理、Spring Boot 父依赖）

拿到输出后，重点检查三件事：

1. 模块声明和目录结构是否一致。父 pom 里声明的 `<modules>` 列表和实际创建的目录要完全对应，一个不多一个不少。
2. 依赖关系是否正确。hify-chat 的 pom 里应该有对 hify-agent 和 hify-provider 的依赖，hify-agent 的 pom 里应该有对 hify-mcp 的依赖。交叉检查，不要有多余的依赖，也不要有遗漏。
3. 版本管理是否统一。Spring Boot、MyBatis-Plus、Redis 的版本号应该只出现在父 pom 的 `<dependencyManagement>` 里，子模块不重复声明版本。

这些都没问题，第一步完成。有问题让 Claude Code 改，不要手动改，让它按规范输出。

第二步：hify-common 公共基础设施

这一步是重中之重。hify-common 里的东西会被所有业务模块依赖——Result 类、异常处理、MyBatis-Plus 配置。这里出了问题，后面每个模块都会跟着错。

我把这一步进一步拆成五个小任务。前三个有依赖链（异常处理器依赖 Result 和 BizException），后两个相对独立。顺序还是按依赖关系排。

任务一：统一响应 Result 和分页 PageResult

Cluade 的指令是：

> 在 hify-common 中创建统一响应类。按照 CLAUDE.md 接口规范：Result<T> 包含 code、message、data 三个字段，提供 ok() 和 fail() 静态方法。PageResult<T> 继承 Result，额外包含 total、page、size。

输出是：



这个任务简单、边界清晰，Claude Code 基本不会出错。

任务二：错误码枚举和业务异常

Cluade 的指令是：

> 在 hify-common 中创建错误码枚举 ErrorCode 和业务异常类 BizException。ErrorCode 包含 code 和 message，覆盖通用错误（参数错误、未授权、系统内部错误等）。BizException 持有 ErrorCode，支持自定义 message 覆盖。

输出是：

两个文件已创建：

- ErrorCode.java — 枚举，按模块分段，覆盖 1000-7999 全段
- BizException.java — 持有 `ErrorCode`，三个构造器：

构造器	场景
`BizException(errorCode)`	直接用枚举默认 message
`BizException(errorCode, message)`	自定义 message 覆盖（如带上具体 id）
`BizException(errorCode, cause)`	包装底层异常（如超时、调用失败）

用法示例：

```
throw new BizException(ErrorCode.PROVIDER_NOT_FOUND);
throw new BizException(ErrorCode.PROVIDER_NOT_FOUND, "Provider id=42 不存在");
throw new BizException(ErrorCode.PROVIDER_CONNECTION_FAILED, e);
```

任务三：全局异常处理器

Cluade 的指令是：

> 在 hify-common 中创建全局异常处理器 GlobalExceptionHandler，使用 @RestControllerAdvice。捕获 BizException 返回对应错误码，捕获 MethodArgumentNotValidException 返回参数校验错误，兜底捕获 Exception 返回系统内部错误。所有异常响应必须使用 Result.fail() 和 ErrorCode 枚举。

输出是：

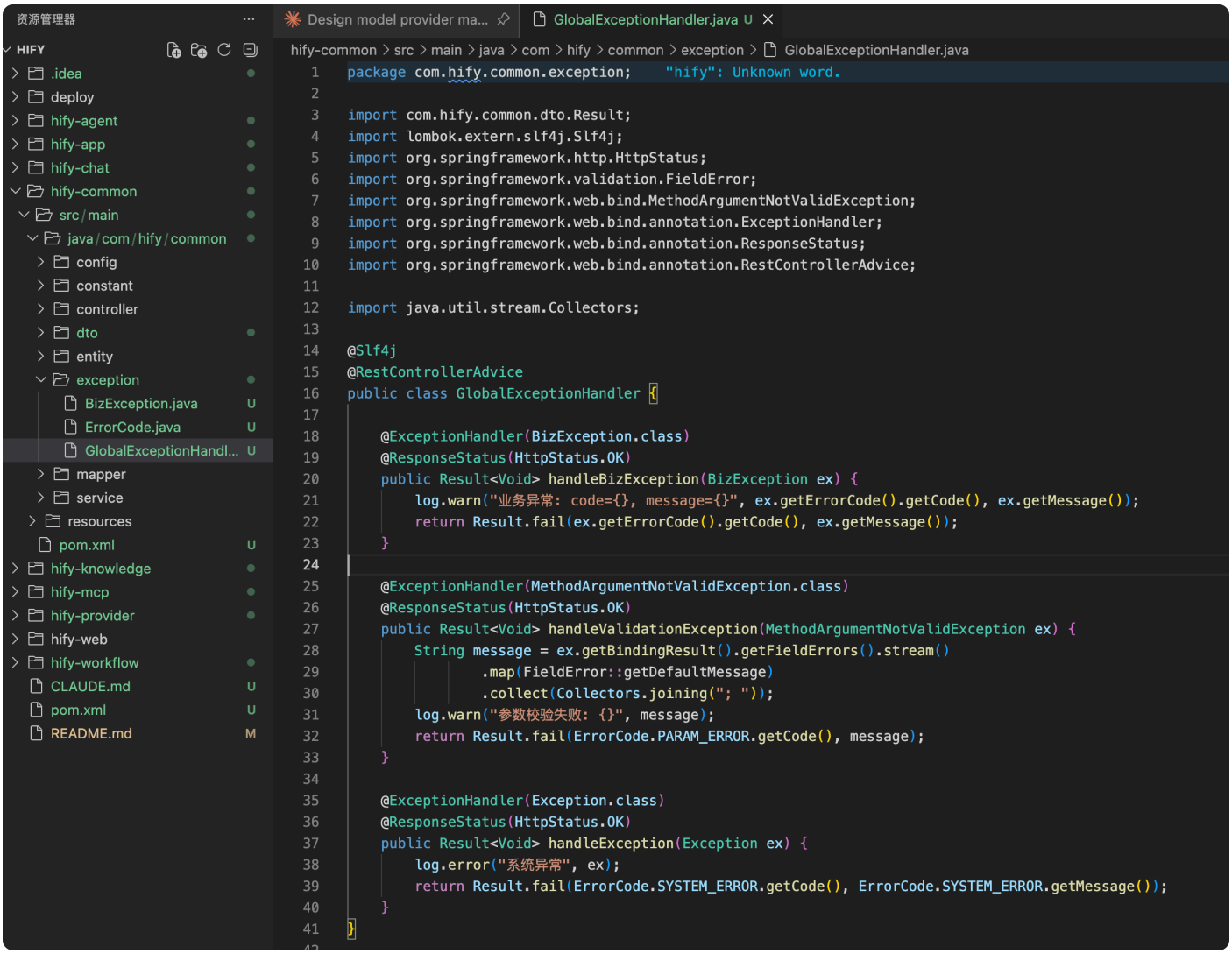
GlobalExceptionHandler.java 已创建，三个处理器：

捕获	HTTP 状态	响应 code	日志级别
`BizException`	200	`errorCode.getCode()`	WARN
`MethodArgumentNotValidException`	200	1000（`PARAM_ERROR`）	WARN
`Exception`（兜底）	200	1999（`SYSTEM_ERROR`）	ERROR（含堆栈）

两点说明：

1. **HTTP 状态统一 200** — 错误信息通过 `Result.code` 区分，符合 CLAUDE.md 的统一响应规范，避免前端同时处理 HTTP 状态码和业务码两套逻辑
2. **兜底只返回 `SYSTEM_ERROR` message** — 不把内部异常信息透传给前端，避免信息泄露；完整堆栈记录在 ERROR 日志中

这个任务值得多说一点。我第一次让 Claude Code 生成这个处理器的时候，它在兜底的 Exception 处理里硬编码了 `code: 500, message: "系统繁忙"`，没有用 ErrorCode 枚举。功能上没问题，但违反了 CLAUDE.md 里定义的规范——所有错误响应必须走 ErrorCode。



这就是 SDD 闭环在起作用的时刻。我让它改成 `Result.fail(ErrorCode.INTERNAL_ERROR)`，然后在 CLAUDE.md 的行为指令里补了一条：“异常处理必须使用 ErrorCode 枚举，禁止硬编码错误码和错误信息。” 下次再让它写类似代码，这个问题就不会重复出现。

每次 AI 跑偏，不只是改掉当前的错，更要把规范补上，堵住同类问题的口子。这就是 SDD 闭环的日常运转——定规范、AI 执行、发现偏差、迭代规范。不是什么宏大的流程，就是写代码过程中随手做的事。

任务四：MyBatis-Plus 配置

Cluade 的指令是：

> 在 hify-common 中创建 MyBatis-Plus 配置类。包含：分页插件、自动填充（createTime、updateTime）、逻辑删除配置。

输出是：

创建完毕：

MyBatisPlusConfig.java

- **分页插件**：`PaginationInnerInterceptor(DbType.MYSQL)`
- **自动填充**：insert 时填 `createdAt`、`updatedAt`、`deleted=0`；update 时填 `updatedAt`

application.yml

- **逻辑删除**全局配置，`deleted=1` 表示删除，`deleted=0` 表示正常

Entity 使用时只需加对应注解：

```
@TableField(fill = FieldFill.INSERT)
private LocalDateTime createdAt;

@TableField(fill = FieldFill.INSERT_UPDATE)
private LocalDateTime updatedAt;

@TableLogic
private Integer deleted;
```

这里只做配置层面的基础搭建。具体的业务封装——BaseEntity 基类、分页查询工具类——在基础组件篇展开。这里想说一下，Claude 生成这种模板代码非常快，基本不会出错，因为这些代码是非常标准的。

任务五：Redis 配置

Cluade 的指令是：

> 在 hify-common 中创建 Redis 配置类。包含：RedisTemplate 序列化配置（key 用 String，value 用 JSON）、基础的 RedisUtil 工具类（get/set/delete/expire）。

输出是：

两个文件已创建：

RedisConfig.java

- key/hashKey → `StringRedisSerializer`
- value/hashValue → `Jackson2JsonRedisSerializer`，带 `JavaTimeModule`（支持 `LocalDateTime`）
- 开启 `activateDefaultTyping`，JSON 中写入类型信息，反序列化时能还原正确类型

RedisUtil.java

方法	说明
`set(key, value)`	永久存储
`set(key, value, ttl)`	带过期时间存储
`get(key)`	返回 `Optional<T>`，避免调用方空指针
`delete(key)`	删除
`expire(key, ttl)`	设置/更新过期时间
`hasKey(key)`	判断 key 是否存在

用法示例：

```
redisUtil.set("session:abc", sessionObj, Duration.ofHours(2));
Optional<ChatSession> session = redisUtil.get("session:abc");
redisUtil.expire("session:abc", Duration.ofHours(2));
```

同样，这里只做配置。Cache-Aside 模式的业务封装在基础组件篇再做。

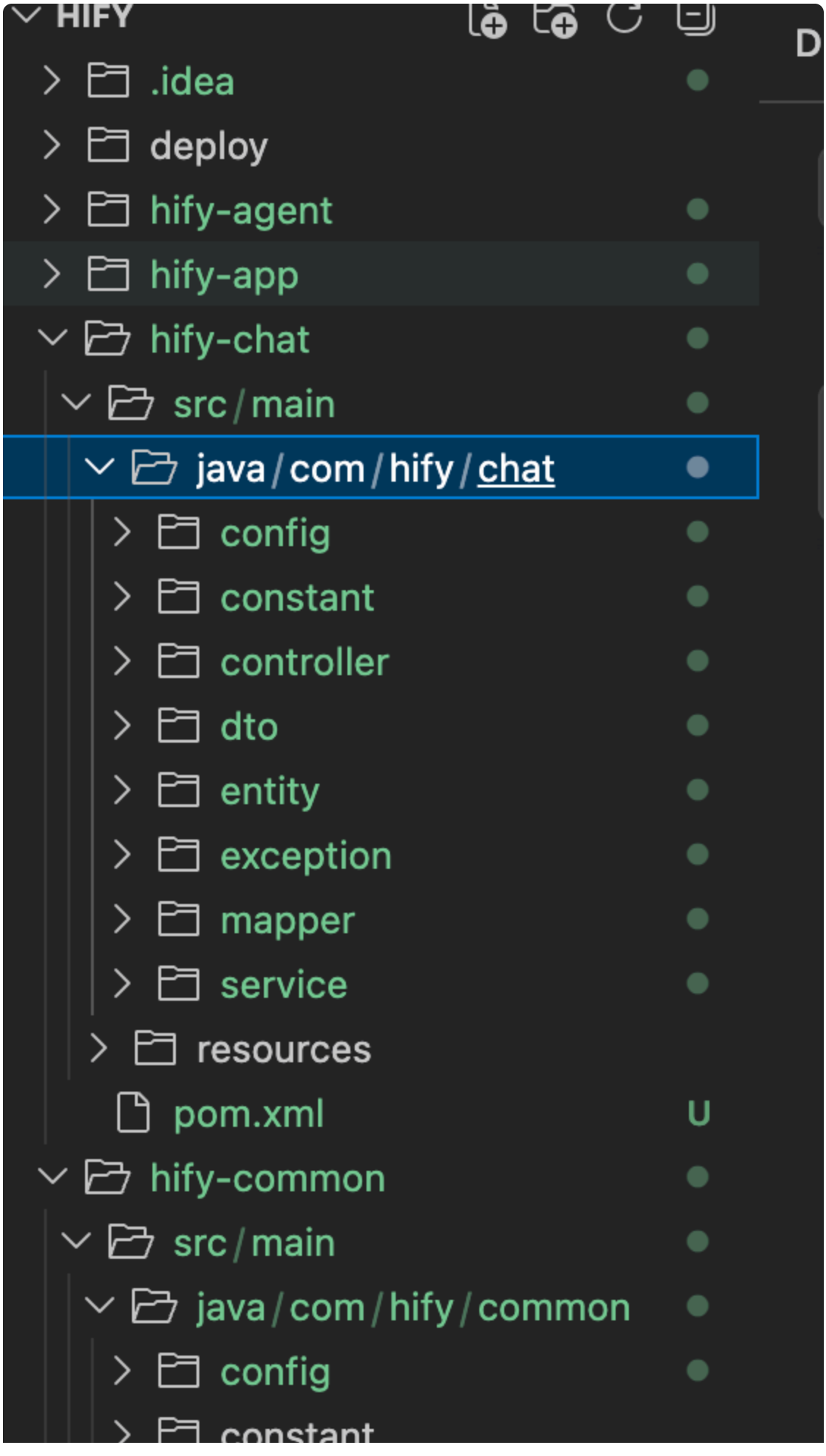
你可能注意到了，我把 hify-common 拆成了五个小任务，而不是一条指令“帮我把 hify-common 全部搭好”。原因和前面讲的一样：拆开做，**每个任务的上下文更小、更聚焦，Claude Code 的输出质量更高。**而且出了问题容易定位，混在一起生成，一个地方有问题可能要在几百行代码里找。

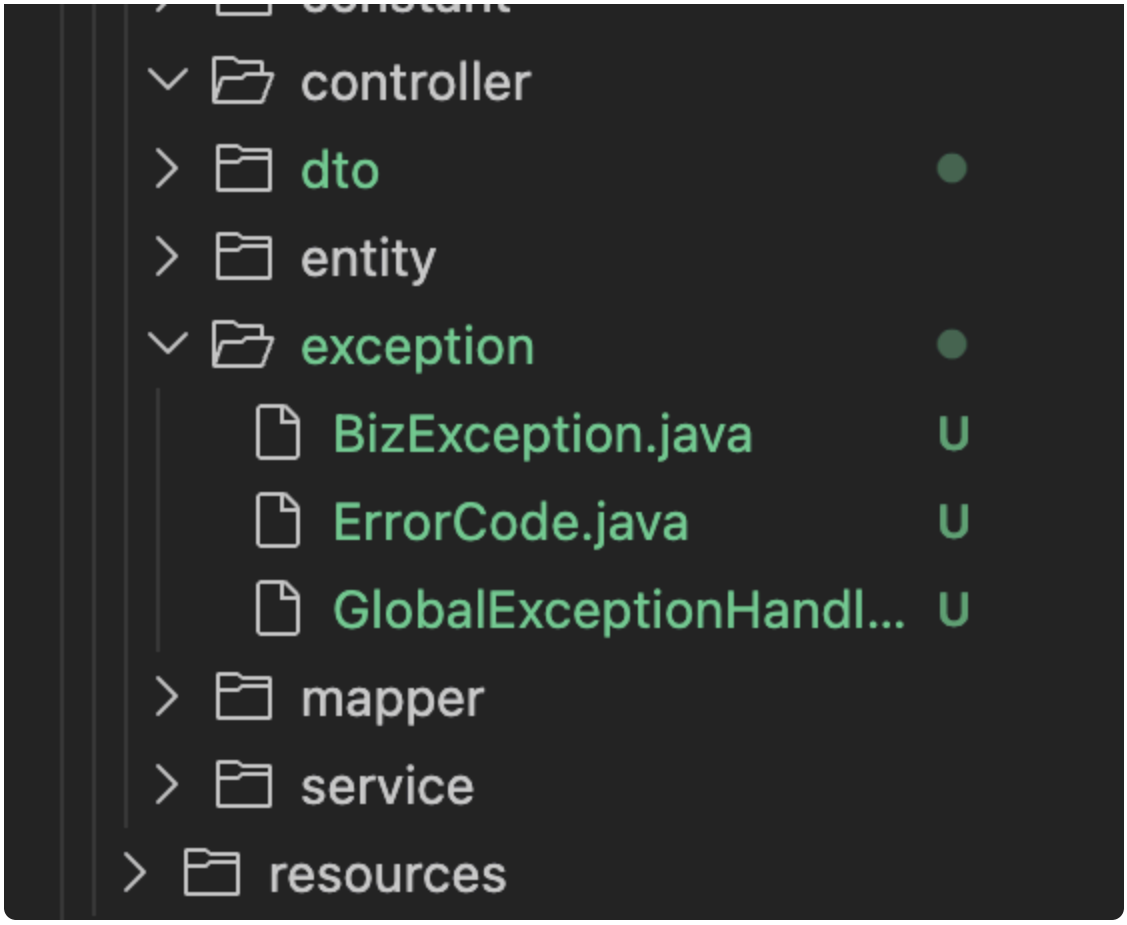
第三步：业务模块空壳

前两步搞定后，后端的地基就打好了。现在给每个业务模块创建基础的 package 结构。

> 为 hify-provider、hify-agent、hify-chat、hify-mcp 等业务模块创建标准的 package 结构。按照 CLAUDE.md 代码组织规范，每个模块包含 controller/service/service-impl/mapper/entity/dto/config 目录。每个模块只创建 package 和一个空的占位类，不需要写业务代码。

这一步生成的东西很简单，review 也快——检查 package 路径对不对、各模块结构是否一致就行。


HIFY
.idea
deploy
hify-agent
hify-app
hify-chat
src/main
java/com/hify/chat
config
constant
controller
dto
entity
exception
mapper
service
resources
pom.xml
U
hify-common
src/main
java/com/hify/common
config
constant



然后在 hify-app 模块里创建 Spring Boot 启动类和 application.yml：

> 在 hify-app 中创建 Spring Boot 启动类 HifyApplication，以及 application.yml 配置文件。配置项包括：数据库连接、Redis 连接、MyBatis-Plus 配置、服务端口 8080。

两个文件已创建/更新：

HifyApplication.java

- `scanBasePackages = "com.hify"` — 扫描所有子模块的 Spring Bean
- `@EnableJpaAuditing` — 启用 MyBatis-Plus 自动填充（`createdAt` / `updatedAt`）

application.yml

配置项	说明
数据库连接	支持环境变量覆盖（`DB_HOST` / `DB_PORT` / `DB_NAME` / `DB_USERNAME` / `DB_PASSWORD`），默认连本地
Redis 连接	同样支持环境变量，默认本地 6379
HikariCP	连接池 5~20，超时 30s
Lettuce 连接池	2~10 连接
MyBatis-Plus	下划线转驼峰、mapper xml 路径、逻辑删除、id 自增

本地开发直接 `java -jar` 启动即用默认值；容器部署时通过环境变量注入真实连接信息，不需要改配置文件。

第四步：验收

后端骨架应该搭好了。启动验证一下。

先确保 MySQL 和 Redis 在本地跑着（如果本地没有，用 Docker 临时起一个，这不是项目的 Docker 化，只是本地开发环境）。

```
cd hify
mvn clean install -DskipTests
cd hify-app
mvn spring-boot:run
```

复制代码

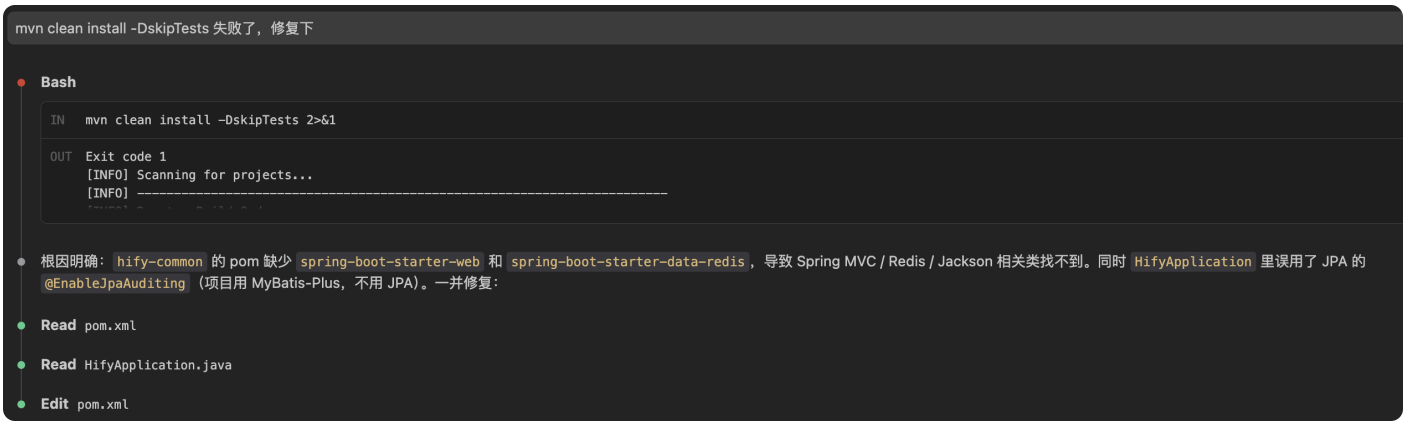
然后，我 mvn clean install 就报错了：

```
/Desktop/hify/hify-common/src/main/java/com/hify/common/exception/GlobalExceptionHandler.java:[21,9] 找不到符号
量 log
com.hify.common.exception.GlobalExceptionHandler
/Desktop/hify/hify-common/src/main/java/com/hify/common/exception/GlobalExceptionHandler.java:[21,49] 找不到符号
法 getErrorCode()
为com.hify.common.exception.BizException的变量 ex
/Desktop/hify/hify-common/src/main/java/com/hify/common/exception/GlobalExceptionHandler.java:[22,30] 找不到符号
法 getErrorCode()
为com.hify.common.exception.BizException的变量 ex
/Desktop/hify/hify-common/src/main/java/com/hify/common/exception/GlobalExceptionHandler.java:[31,9] 找不到符号
量 log
com.hify.common.exception.GlobalExceptionHandler
/Desktop/hify/hify-common/src/main/java/com/hify/common/exception/GlobalExceptionHandler.java:[32,49] 找不到符号
法 getCode()
为com.hify.common.exception.ErrorCode的变量 PARAM_ERROR
/Desktop/hify/hify-common/src/main/java/com/hify/common/exception/GlobalExceptionHandler.java:[38,9] 找不到符号
量 log
com.hify.common.exception.GlobalExceptionHandler
/Desktop/hify/hify-common/src/main/java/com/hify/common/exception/GlobalExceptionHandler.java:[39,50] 找不到符号
法 getCode()
为com.hify.common.exception.ErrorCode的变量 SYSTEM_ERROR
/Desktop/hify/hify-common/src/main/java/com/hify/common/exception/GlobalExceptionHandler.java:[39,84] 找不到符号
法 getMessage()
为com.hify.common.exception.ErrorCode的变量 SYSTEM_ERROR
```

然后我继续输入：

> mvn clean install –DskipTests 失败了，修复下

输出是：



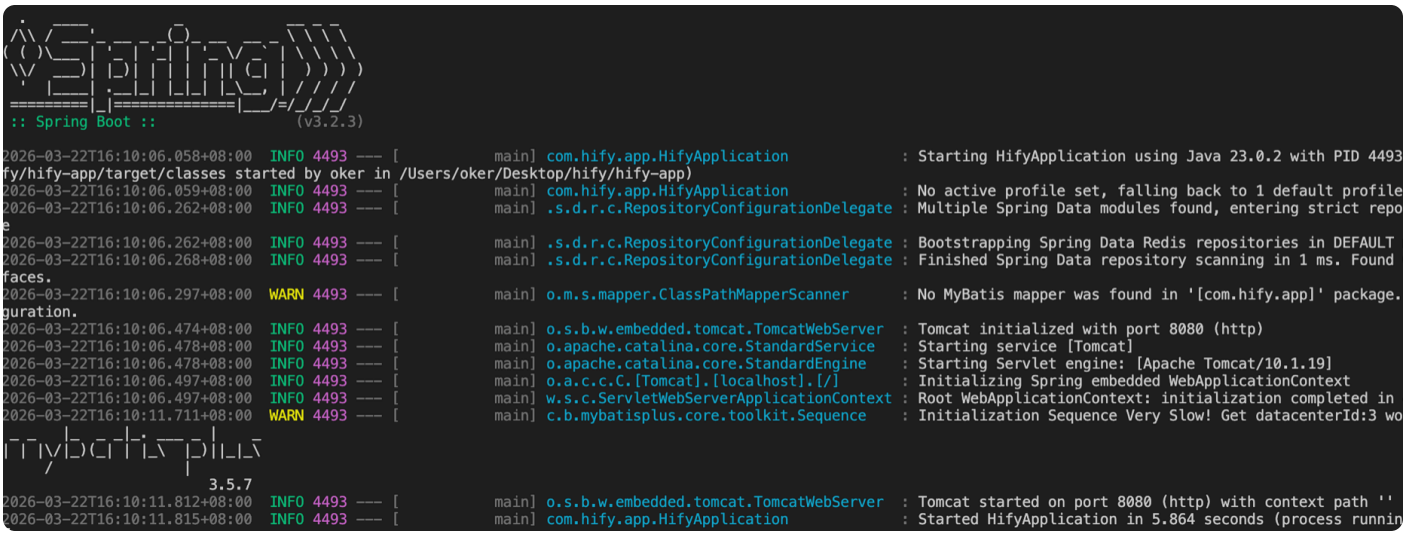
然后修复这个问题又失败了。



那就继续让 Claude Code 继续修复。这个过程重复，就可以得到一个可以启动的应用了。

这里一直起不来，后来 Claude Code 发现前面给的启动命令错了（哈哈），应该是下面这个命令：

```
mvn spring-boot:run -pl hify-app
```



Spring Boot 启动日志的最后一行应该显示 `Started HifyApplication in X seconds`。

为了有一个可验证的端点，让 Claude Code 加一个健康检查接口：

在 hify-app 中创建 HealthController，路径 GET /api/v1/health，返回 Result.ok("Hify is running")。

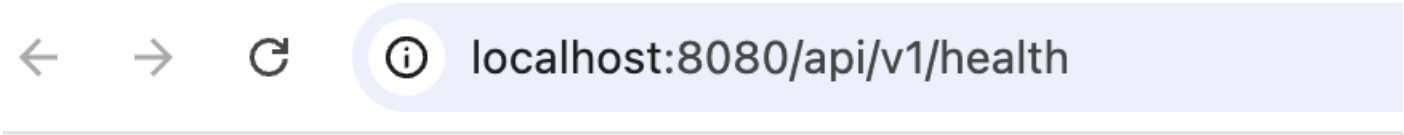
启动后访问 `http://localhost:8080/api/v1/health`，应该可以看到：

复制代码

```
{
  "code": 200,
  "message": "success",
  "data": "Hify is running"
}
```

看到这个返回，说明后端骨架搭建成功。Maven 多模块结构正常、依赖关系正确、公共模块的 Result 和全局异常处理在工作、Spring Boot 配置没问题。

但是输出却是：




```
{
    "code": 1999,
    "message": "系统内部错误",
    "data": null
}
```


接下来就交给你了。



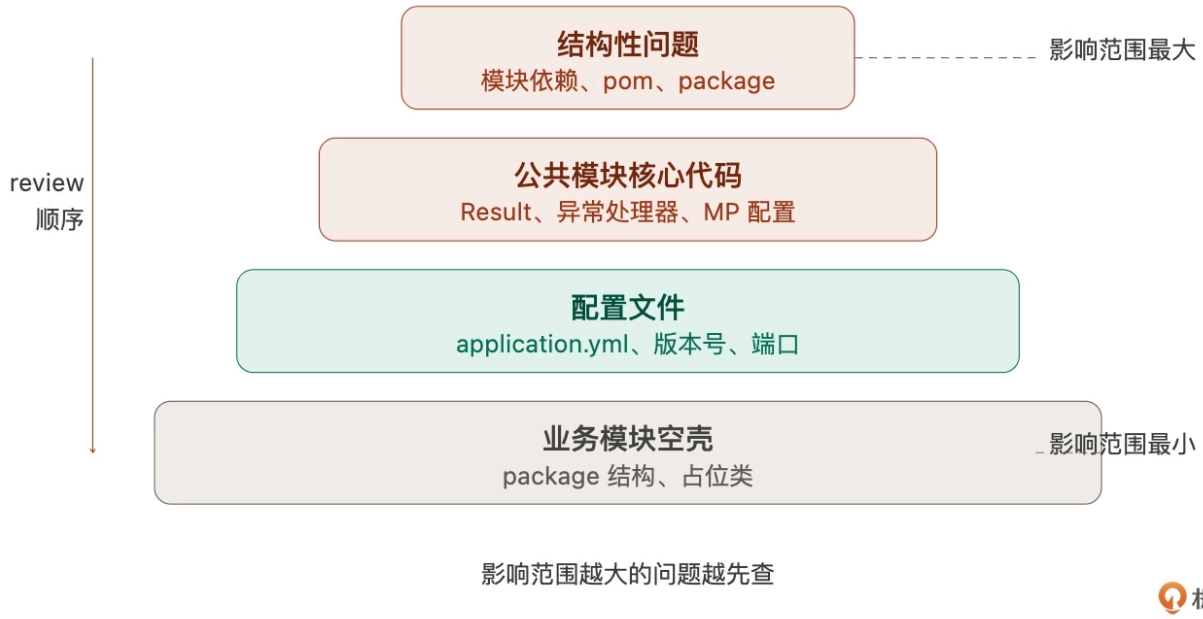

```
{
    "code": 200,
    "message": "success",
    "data": "ok"
}
```


大批量代码怎么 review

这一讲 Claude Code 生成了不少代码，几十个文件、几百行配置和基础代码。逐行看不现实，也没必要。关键是按优先级看。

1. 第一优先级：结构性问题。模块依赖关系对不对？pom 里的依赖声明和 CLAUDE.md 里定义的架构一致吗？package 路径对不对？这些如果错了，后面所有东西都建在错误的地基上。
2. 第二优先级：公共模块的核心代码。Result 类的字段和方法对不对？全局异常处理器的捕获优先级对不对？MyBatis-Plus 的自动填充逻辑对不对？这些代码所有业务模块都会依赖，错了影响范围最大。
3. 第三优先级：配置文件。application.yml 里的配置项对不对？有没有遗漏？有没有硬编码不该硬编码的东西？影响范围相对小，后面随时可以调整。

4. 最后才看：业务模块空壳。只是 package 结构和占位类，几乎没什么可出错的，扫一眼确认结构一致就行。



这个优先级背后的逻辑很简单：**影响范围越大的问题越先查**。结构错了全盘皆输，公共模块错了所有业务模块跟着错，配置错了这个服务有问题，空壳错了只影响一个文件。把 review 时间花在影响范围最大的地方。

这个思路不只适用于工程初始化。后面每一讲 Claude Code 生成大量代码时，都可以用同样的方式。先找影响范围最大的问题，再逐步往下。

总结

这一讲做了两件事：用 Claude Code 搭建了 Hify 的后端骨架，以及总结了 3 个后面会反复使用的方法论。

任务拆解标准。生成量是否超出一次 review 的范围？步骤之间是否有依赖关系？满足其一就拆。拆的顺序按依赖关系来，先地基后框架。

影响范围 review。大批量代码不需要逐行看。按影响范围排优先级——结构 > 公共模块 > 配置 > 空壳，时间花在影响范围最大的地方。

每步验收。不是搭完全部才验收，而是每一步做完就验证。Maven 骨架搭好了 `mvn install` 能通过吗？公共模块写好了编译没问题吗？全部搭完 `spring-boot:run` 能启动吗？每一步的验收给你信心：“到这里为止是对的。”不验证就往前冲，错误会累积，到最后都不知道问题出在哪一步。

到了这里我们依然 **0 代码**，就完成了一个 Web Service 的初始化。中间基本没遇到问题，很顺。一方面有模型厉害的原因，一方面是我们前面基础搭建的好，一方面是我们把一个大任务拆解为多个小任务了。在后面的课程，你会越来越感受到这个模式的优势。

你可以思考下，如果我们自己来做这些事情，是不是至少要一天。而有了 AI 辅助后，可以又快又好的实现。节省下来的时间，就可以用来做其他事情，这就是生产力的提升。

后端能跑了。下一讲补前端 Vue 工程、写启动脚本，让 Hify 的前后端一键跑起来。

思考题

回想一下你之前做项目时的工程初始化过程。你是手动一个个文件创建，用脚手架工具，还是从别的项目复制？如果现在让你用 Claude Code 来做，你会怎么拆解？试着列出你的步骤和每步的验收标准。

期待你与我分享自己的使用体验。如果今天的课程让你有所收获，也欢迎转发给有需要的朋友，邀请他来一起学习，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (17)

最新　精选

星辞Sting
2026-05-02 来自浙江

大佬，目前各个大厂都在做AI CODING，也希望能够颠覆原有的开发流程。 比如编码Agent\测试Agent\产品Agent\运维Agent等等，用AI来做全链路自动化智能化。 按照大佬课上的分小功能点逐一实现，那这样这个全流程AI自动化多久能实现？ 也从大厂同学里打听到他们都是管着Agent了，这个目前的风险点和突破口等你怎么看

作者回复: 那这样这个全流程AI自动化多久能实现？ 也从大厂同学里打听到他们都是管着Agent了，这个目前的风险点和突破口等你怎么看

_____

个人粗浅的见识。

1. 全流程AI自动化多久能实现。已经很多公司在做了，包括我们自己。但是愿景是“全流程AI自动化实现”，愿景特别好，大家都认可。但是现实是投入了很多很多很多人力，效果嘛，我看到的比想象中差特别特别多。

2. 从大厂同学里打听到他们都是管着Agent了。 咋说呢，我个人见到的“他们都是管着Agent”这句话有夸大的成分了。很多业务流程慢慢在用Agent流程化，但是说实话，落地效果没有这句话这么大。至少我看到的是这样

3. “这个目前的风险点和突破口等你怎么看”我觉得未来会走到这一步的，很多流程会被Agent 化，但是需要时间。现在处于大家在摸索投入搭建基础设施的阶段，我预计一两年吧，等各个公司或开源的生态（比如最近的Hermes Agent、OpenClaw）的完善。我觉得很多流程会自动化，agent化的。

总结就是：当前没说的那么好，未来我觉得会到这一步。

共 2 条评论 >

6

Geek_7740b8

2026-05-19 来自北京

这门课不仅仅是AI coding的课，老师还带着把一个项目的很多设计考量讲了，哪怕抛开AI编程部分，对我们这种spring mvc新手来说很多内容都太值了

作者回复: 感谢感谢～～🌹

5

peterpc

2026-04-18 来自云南

老师，能否提供一个基于已有项目进行功能新增的全流程使用案例

作者回复: 可以的，已经在规划中了～

共 3 条评论 >

1

kobe.鹏
2026-06-29 来自上海

老师，在做好基础的claude.md的前提下，每个小任务是不是可以按照openspec+superpowers的形式执行啊

作者回复: 是的，可以的。前提就是你要设计好、拆好任务，再openspec+superpowers去执行。 或者你可以试用下spec kit。让spec kit 给你拆分好任务。然后执行。
不过如果是重要项目，需要注意小步快跑，不要一下子让ai写太多，自己代码最好都要看。






庄周梦蝶
2026-06-24 来自浙江

老师，请教：感觉也可以用插件来引导claude code一次性输出执行计划（它也会找你确认），让后输出任务（能识别出哪些任务是有先后顺序 哪些有依赖关系）， 然后它会一个一个的完成，最后还可以自己验收测试（计划中列出了验收标注的话）。老师你认为我这样的想法是合理且现在能实现的吗？ 对于经验不足的人 本身就没能力想到那么多 所以不可能一步步的提。

作者回复: 你这样是合理的。近期的loop engineering 就是在讲这个事情。
只是你需要区别这种场景对你的价值是什么：
1. 比如你是想快速做出东西，那么这个方法基本可以满足你的需求。能让AI 做出东西。做出来后你就得考虑后续怎么维护、演进、迭代。
2. 比如你是想学习，这种方法跑出来的代码，你得去思考ai 为什么这么拆任务，代码为什么这么写。
所以，你的思路是可以做出东西的，但是你得看你这么做的目的是什么。






未来已来
2026-05-28 来自广东

图片不能放大😂

作者回复: 右键 – 新标签也打开图片是不是可以～






Peter
2026-05-18 来自山西

课程中的图片用什么画的？如果使用的AI，具体是哪个工具？

> 作者回复: 我用的是AI画的。用的claude ，用的是这个：https://time.geekbang.org/column/article/975826






pcc
2026-04-28 来自湖北

老师能提供下最新的CLAUDE.md或者git地址吗，我采用07节最后提供的CLAUDE.md与你生成的项目结构不一致。

> 作者回复: 不一样是因为模型的差异或者同一个提示词跑出来的结果不一样导致的。这是项目地址哈：https://github.com/socutes/hify

共 2 条评论>





牛牛
2026-04-28 来自北京

modules/{module}/ ├── api/ # 对外暴露的接口（interface），供其他模块调用 ├── domain/ # 业务逻辑：Service 实现、领域对象、Factory、Repository 接口 ├── infra/ # 基础设施：Mapper、RepositoryImpl、外部 API 客户端、config └── web/ # Controller，只处理 HTTP 层 你写的CLAUDE.MD中的这一段是什么？和你上面截图的包结构对上吗？你的结构图中有这些包吗？

> 作者回复: https://github.com/socutes/hify 这个是项目地址，可以看一下这个。因为我第一次完整做了一遍，然后在演示的时候又从头做了一遍。两次之间可能代码目录结构有点差别，但是不影响功能实现。






木子皿
2026-04-26 来自浙江

 作为c++开发者，只能学习核心理念了

作者回复: 要不用这个思路尝试下C++实现。 我感觉提示词可能差不多。哈哈哈。前面有个同学要用python。我蛮期待的。要不咱们用C++试试。我觉得AI 屏蔽了很多语言的区别。可能没想象中那么难。



